Namespace

A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it. Namespaces are used to organize code into logical groups and to prevent name collisions that can occur especially when your code base includes multiple libraries.

Multiple namespace blocks with the same name are allowed. All declarations within those blocks are declared in the named scope.

With Scope resolution operator (::)

Any name (identifier) declared in a namespace can be explicitly specified using the namespace's name and the scope resolution :: operator with the identifier.

```
// Creating namespaces

#include <iostream>
using namespace std;
namespace ns1
{
     int value(){ return 5; }
}
namespace ns2
{
     const double x = 100;
     double value() { return 2*x; }
}

int main()
{
     // Access value function within ns1
     cout << ns1::value() << '\n';

     // Access value function within ns2
     cout << ns2::value() << '\n';

     // Access variable x directly
     cout << ns2::x << '\n';
```

Vector India

```
    return 0;
}
Output:
5
200
100
```

The functionality of namespaces is especially useful in the case that there is a possibility that a global object or function uses the same identifier as another one, causing redefinition errors. For example:

```
#include<iostream>

using namespace std;

namespace first

{

int var=5;

}

namespace  second

{

double var=3.14;

}

int main()

{

cout<<first::var<<endl;

cout<<second::var<<endl;

}
```

Vector India

Output:

5

3.14

In this case, there are two global variables with the same name: var. One is defined within the namespace first and the other one in second. No redefinition errors happen thanks to namespaces.

using

The keyword using is used to introduce a name from a namespace into the current declarative region. For example:

```
#include<iostream>
using namespace std;
namespace first
{
	int x=5;
	int y=10;
}
namespace second
{
	double x=3.14;
	double y=2.71;
}
int main()
{
	using first :: x;
	using second ::y;
	cout<<x<<endl;
	cout<<y<<endl;
```

Vector India

```
    cout<<first::y<<endl;
    cout<<second::x<<endl;
}
```

Output:

5

2.71

10

3.14

Notice how in this code, `x` (without any name qualifier) refers to `first::x` whereas `y` refers to `second::y`, exactly as our `using` declarations have specified. We still have access to `first::y` and `second::x` using their fully qualified names.

The keyword using can also be used as a directive to introduce an entire namespace:

Example:

```
#include<iostream>
using namespace std;
namespace first
{
    int x=5;
    int y=10;
}
namespace second
{
    double x=3.14;
    double y=2.71;
}
using namespace first;
int main()
```

Vector India

```
{
        cout<<x<<endl;
        cout<<y<<endl;
        cout<<second::x<<endl;
        cout<<second::y<<endl;
}
```

Output: 5
10
3.14
2.71

In this case, since we have declared that we were `using namespace first`, all direct uses of x and y without name qualifiers were referring to their declarations in `namespace first`.

Namespace alias

We can declare alternate names for existing namespaces according to the following format:

```
namespace new_name = current_name;
```

Anonymous or unnamed namespaces

You can create an explicit namespace but not give it a name:

```
namespace
{
    int MyFunc(){}
}
```

This is called an unnamed or anonymous namespace and it is useful when you want to make variable declarations invisible to code in other files (i.e. give them internal linkage) without having to create a named namespace. All code in the same file can see the identifiers in an unnamed namespace but the identifiers, along with the namespace itself, are not visible outside that file—or more precisely outside the translation unit.

Vector India

The std namespace

All C++ standard library types and functions are declared in the `std` namespace or namespaces nested inside `std`.

Nested namespaces

Namespace inside a namespace is called nested namespace

example:

```
#include<iostream>
using namespace std;
namespace ABC
{
    int data=20;
     namespace XYZ
     {
          float f=3.56;
     }
}
main()
{
    cout<<ABC::data<<endl;
    cout<<ABC::XYZ::f<<endl;
}
```

Output

```
20
3.56
```

Vector India